

12. Pruebas y calidad

PDF Reader v0.3.0 | Análisis 2026-09-10 | Commit 265b122

Contenido

- Gates actuales
- Resultado de esta ejecución
- Casos cubiertos
- Cobertura faltante priorizada
- Análisis estático y formato
- Criterios de aceptación
- Datos de prueba

Gates actuales

Gate	Herramienta	Cobertura observable
Unitarias y contratos Web	Node --test	21 casos de utilidades, interacción, ciclo de vida y almacenamiento defensivo
Contrato repositorio	verify-repo.mjs	estructura, alcance read-only, zoom, versión, permisos y workflows
Build Web/Pages	scripts Node	copia PDF.js y composición landing+demo
Android unit	Gradle/JUnit	prueba trivial de plantilla
Android instrumentada	AndroidX	application ID
CI Web	Ubuntu	install, check, audit, build y artifact
CI Android	Ubuntu/JDK/SDK	sync limpio, tests, APK debug y metadatos/permisos
CI Windows	Windows	instalador NSIS y portable
Release	Ubuntu+Windows	versión, firma APK, contenido y checksums
Documentación integral	verify-system-docs.mjs	20 Markdown/PDF, enlaces, tablas y UTF-8

Resultado de esta ejecución

El 2026-09-23 se aplicó la suite preliminar de producto sobre Web local. La ejecución fresca posterior a las correcciones obtuvo:

Bloque de código

```
CI=true pnpm test           -> 21/21 pruebas
CI=true pnpm release:check  -> test, verify, docs y build Pages correctos
pnpm build:web              -> dist generado
```

La prueba funcional abrió un PDF desde Historial y luego un segundo PDF sin recargar. Antes de la corrección el segundo paso fallaba con `previousDoc.destroy is not a function`; después, el segundo documento quedó visible sin errores nuevos. El informe trazable es `reports/preliminary-product-validation-2026-09-23.md`.

Referencia histórica de la documentación integral

El 2026-09-10 se ejecutó:

Bloque de código

```
CI=true pnpm install --frozen-lockfile -> correcto, lockfile sin cambios
```

```
pnpm check          -> 9/9 pruebas, verify OK
pnpm build:pages     -> dist y pages-dist generados
pnpm docs:check      -> estructura, enlaces, tablas, UTF-8 y PDF correctos
pnpm exec node scripts/run-gradle.mjs test -> BUILD SUCCESSFUL, 138 tareas
```

El primer intento sin `CI=true` no ejecutó tests porque pnpm no podía confirmar de forma interactiva la recreación de `node_modules`. Gradle requirió seleccionar explícitamente el JDK 21 de Android Studio y el SDK local; con ambos activos, las variantes debug/release y sus pruebas terminaron correctamente.

Casos cubiertos

`utils.test.mjs` valida cálculos, formato, snippets seguros, identidad, gestos, compartir, ancla de zoom y swipe. `app-contract.test.mjs` protege navegación, documentos grandes, segunda apertura, concurrencia y guardado en cambios de ciclo de vida. `storage.test.mjs` comprueba lectura/escritura normal y degradación ante `SecurityError` o cuota. `verify-repo` confirma marcadores del layout, ausencia de funciones de edición, versión Android, permisos prohibidos, asociación PDF, release firmado y Pages.

Cobertura faltante priorizada

- 1 **Alta:** automatizar en CI los recorridos DOM/E2E hoy comprobados manualmente: apertura sucesiva, navegación, render, búsqueda, historial, errores y teclado.
- 2 **Alta:** pruebas Android reales para intent, pinza, rotación, background/restore y compartir.
- 3 **Alta:** pruebas Electron del preload/IPC, asociación `.pdf` y rutas inválidas.
- 4 **Media:** IndexedDB: poda a ocho, transacciones fallidas, cuota y migraciones; `localStorage` bloqueado sí dispone de regresiones unitarias.
- 5 **Media:** PDF cifrado, corrupto, enorme, sin texto, con CMaps y muchas páginas.
- 6 **Media:** accesibilidad automatizada y manual con lector de pantalla.
- 7 **Baja:** reemplazar `additionIsCorrect` por pruebas del plugin nativo.

Análisis estático y formato

No se identificaron ESLint, Prettier, TypeScript, cobertura, SAST ni escáner de secretos dedicados. Node interpreta ESM/CJS durante tests/build. Los workflows ejecutan `pnpm audit --prod --audit-level high`; Dependabot revisa npm y Actions semanalmente.

Criterios de aceptación

El `spec/spec.md` exige tests Node, verificación, build Web, packaging Windows, APK debug inspeccionado, APK release firmado, documentación, Pages y release por tag. Para considerar una versión publicable deben estar verdes los tres checks protegidos de main: Web · tests + landing, Windows · package y Android · APK debug verificado.

Datos de prueba

No se versionan PDF privados. `scripts/create-smoke-pdf.mjs` crea un PDF sintético de una página. Los informes `reports/validation-v0.*.md` conservan evidencia histórica, no sustituyen una ejecución actual.